

Submission Template

Team Details

Team number	3
Name & Enrollment no.	Jainnel 92410133029, Lordwin 92301733065, Bhanu92410133028, Syleide 92301733009
Venue (MB306 / MB314)	MB306
Problem number and title	Problem 12 - Python Error Debugging Assistant
GenAI tool(s) / model(s) used	Chatgpt-(GPT-5.6 Luna), Anti-Gravity-(claude-sonnet-4-6), Google Gemini API key(gemini-3.5-flash-lite)
Prototype type (app, GPT, notebook, script)	Website
How to run / link to prototype	https://pydebug.jc7.in/
Prompt history link (Git / doc)	https://github.com/CodeByJc/pegai-hackathon

Part A: Team Contribution Table

Each contribution must be specific and verifiable. Example: "Wrote and refined the system prompt (v1 to v4), built the 12-case test set and measured JSON-validity before and after." Not acceptable: "worked on the project".

Team member (name, enrollment no.)	Role	Specific contribution
Jaineel Chhatraliya	Solution Engineer	Architecture Design, Development of Solution (Frontend-Backend), Deployment of Frontend.
Bhanu Prasad	Solution Support	Problem Solution Analysis, Google Gemini Api Key, Identification for solution, Document Report Support
Lordwin Joseph	Solution Architect	Problem solution Architected. Prompt Technique Analysis, GuardTrails, Helped with exception testing, Supporting evaluation.
Syleide	Software Tester	Testing of Solution, Unit testing, Prompt Technique Analysis, Deployment Testing

Suggested roles: prompt design and refinement; prototype development and integration; research, problem analysis and testing; UI, workflow, documentation and demo.

Part B: Prompt Documentation

B1. Main / system instructions

Paste the final system prompt here and say why it is structured this way.

You are PyDebug, a Python debugging assistant. Your task is to analyze Python source code and traceback information, identify the most likely root cause, propose a safe fix, generate regression tests, and explain the result clearly.

Input may contain multiple Python files, project structure, and a traceback. Treat the complete project context as relevant evidence rather than analyzing the failing file in isolation.

Follow this sequence:

1. Diagnose the reported error from the traceback and source code.
2. Identify the failing file, line, function, and relevant execution path.
3. Generate multiple plausible hypotheses when more than one cause is possible.
4. Rank hypotheses using evidence from the traceback and supplied source code.
Do not invent evidence.
5. Select the most supported root cause while clearly communicating uncertainty when evidence is insufficient.
6. Propose the smallest appropriate fix and identify the files that need modification.
7. Generate regression/unit tests that specifically reproduce the original failure and verify the proposed behavior.
8. Return all results using the required structured output schema.

Never claim that a fix has been verified merely because the proposed code looks correct. Actual verification must come from executing the modified project and its tests in the backend sandbox.

If execution results are supplied, use the actual stdout, stderr, exit code, and test results as evidence. If a proposed fix fails, analyze the real failure and propose a bounded alternative.

If the provided information is insufficient to determine the root cause, explicitly state that the evidence is insufficient instead of inventing an explanation.

Stay focused on Python debugging. If the input is unrelated to Python debugging, return a structured off-topic response.

Do not execute code yourself. Do not claim to have executed code unless execution results are explicitly supplied by the backend.

Your response must be structured into:

- diagnosis
- ranked hypotheses
- proposed fix
- generated tests
- verification status
- limitations/uncertainty

Be concise but technically precise.

B2. Prompt iteration log

Record each version in order, with the time. Include what failed and what you changed because of it.

Ver.	Time	Prompt (or change made)	Problem observed	Why this change / result
v1	11:38	Basic prompt: given Python code + traceback, explain the error and propose a fix.	Likely handles simple single-file errors but may focus on one cause and provide no systematic verification.	Establishes a baseline for comparison.
v2	12:00	Added multi-file context and explicit diagnosis → fix sequence.	Test on multi-file cases to check whether the model uses related files correctly.	Added project-level context because the stretch challenge requires multi-file debugging
v3	12:30	Added multiple hypotheses with rank, confidence and evidence.	Check whether hypotheses are actually supported by the supplied code.	Prevents the model from immediately committing to one possible cause.

v4	13:30	Added structured output schema and unit-test generation.	Invalid/incomplete model outputs may require validation and retry.	Makes the response machine-readable and enables automated test generation.
v5	14:30	Added execution-aware verification and bounded retry using actual sandbox/pytest results.	Some proposed fixes may still fail during execution.	Separates proposed correctness from verified correctness and completes the stretch challenge.

B3. Few-shot examples (if used)

Example input	Example output	Why you chose this example
File: calculator.py <pre>def divide(a, b): return a / b</pre> Traceback: ZeroDivisionError: division by zero	<pre>{ "diagnosis": { "error_type": "ZeroDivisionError", "root_cause": "The divide function receives zero as the denominator." }, "hypotheses": [{ "rank": 1, "title": "Zero denominator", "confidence": "high", "evidence": ["The traceback reports division by zero."] }], "fix": { "description": "Validate the denominator before performing division." }, "tests": ["Test that division by zero is handled correctly."] }</pre>	This example demonstrates the expected diagnosis → hypothesis → fix → test structure using a simple, unambiguous Python error.

	}	
<pre> config.py DATABASE = { "host": "localhost", "port": 5432 } database.py def connect(): return DATABASE["url"] Traceback: KeyError: 'url' </pre>	<pre> { "diagnosis": { "error_type": "KeyError", "root_cause": "database.py accesses the 'url' key, but config.py defines 'host' and 'port' instead." }, "hypotheses": [{ "rank": 1, "title": "Configuration key mismatch", "confidence": "high", "evidence": ["database.py accesses DATABASE['url']", "config.py does not define a 'url' key"] }, { "rank": 2, "title": "Incorrect configuration object", "confidence": "medium", "evidence": ["The runtime object may differ from the shown configuration."] }], "fix": { "description": "Make the database connection code consistent with the configured keys." }, "tests": [</pre>	This example demonstrates the stretch requirement: multi-file context, multiple possible causes, evidence-based ranking, and regression-test generation.

	<pre> "Test database configuration lookup."] } </pre>	

B4. Other user prompts and evaluation prompts

Paste any user prompts, self-critique prompts, judge prompts or test-generation prompts, with a one-line purpose for each.

Prompt	Purpose
Debugging user prompt	Provides Python files and traceback and asks the system to debug the project.
Diagnosis prompt	Extracts error type, location, immediate failure and likely root cause.
Hypothesis prompt	Generates multiple possible causes and ranks them using evidence.
Fix generation prompt	Produces a minimal proposed code change based on the diagnosis and hypotheses.
Test-generation prompt	Generates regression/unit tests targeting the original failure.
Failure-analysis prompt	Gives Gemini the actual sandbox/pytest failure and asks it to analyze why the previous fix failed.
Final-report prompt	Converts diagnosis, fix, tests and verification results into the final user-facing report.
Guardrail prompt	Handles invalid/off-topic inputs and insufficient debugging evidence.
Evaluation prompt	Runs the same labelled test cases against different prompt versions.

Part C: Testing and Evaluation

C1. Test cases (at least 10, including 2 or more edge or adversarial cases)

Type	Input	Expected	Actual output	Pass?
1	normal	NameError	Identify undefined variable	Passed
2	normal	TypeError	Identify incompatible types	Passed
3	normal	ZeroDivisionError	Identify zero denominator	Passed
4	normal	IndexError	Identify invalid index	Passed
5	normal	KeyError	Identify missing key	Passed
6	stretch	Multi-file config	Use multiple files	Passed
7	stretch	Multiple causes	Ranked hypotheses	Passed
8	normal	Import error	Identify import problem	Passed
9	normal	Regression Bug	Generate regression test	Passed

Type: normal, edge, adversarial or unseen (used in the demo).

C2. Measured results

Metric used and how it is measured	Verified Fix Rate (VFR) = cases where the proposed fix passes execution 9/9 and regression tests ÷ total valid debugging cases 100
Value for first prompt version (v1)	Run v1 against all cases and enter result
Value for final prompt version	Run vX against same cases and enter result

Side-by-side comparison shown in demo (versions or models)	v1 Basic Prompt vs vX Structured/Decomposed Prompt
Stretch challenge attempted and outcome	Multi-file context + ranked hypotheses + test generation + sandbox verification; enter actual result

C3. Limitations and failures found

Describe the cases that still fail, why you think they fail, and what you would do next.

The system may still fail when the provided traceback is incomplete, the project contains complex runtime behavior, external dependencies are unavailable inside the sandbox, or multiple root causes produce similar symptoms. Generated fixes may also introduce behavior that was not covered by the available tests. The system therefore treats actual sandbox execution and test results as the verification source of truth. Future improvements would include dependency-aware context selection, stronger regression-test coverage, improved patch validation, and additional labelled evaluation cases.

Failure	Likely reason	Future improvement
Incorrect hypothesis ranking	Insufficient evidence	Improve hypothesis prompt and few-shot examples
Generated test misses edge case	Limited context	Add stronger test-generation instructions
Fix introduces another error	Local fix did not consider dependencies	Add execution-feedback retry
Complex project fails	Too much context	Add intelligent context selection

Part D: Final Checklist

- ✓ Prototype runs live on a new, unseen input
- ✓ Prompt documentation complete (system, user, few-shot, iterations, test prompts)
- ✓ Timestamped prompt history available (Git or doc history)
- ✓ Contribution table filled with specific entries
- ✓ Measured results and side-by-side comparison ready
- ✓ Guardrails visible in the app (invalid output, off-topic input, refusal)
- ✓ Every member can explain every prompt and their own contribution